

课程笔记

# [LLM Agents F24] Compound AI Systems & DSPy —Omar Khattab

基于公开课程资料整理

Fall 2024



OMAR  
KHATTAB

 databricks

视频作者/频道: Berkeley RDI

发布日期: Fall 2024

视频时长: 约 90 分钟

视频链接: [https://www.youtube.com/playlist?list=PLoROMvodv4r0Y23Y0BoGoBGgQ1zmU\\_MT\\_](https://www.youtube.com/playlist?list=PLoROMvodv4r0Y23Y0BoGoBGgQ1zmU_MT_)

项目主页: <https://github.com/hqh1025/ai-course-notes>

# 目录

|                                       |          |
|---------------------------------------|----------|
| <b>1 引言：从单模型到复合 AI 系统</b>             | <b>2</b> |
| 1.1 本章小结 . . . . .                    | 2        |
| <b>2 Compound AI Systems 的范式</b>      | <b>2</b> |
| 2.1 什么是 Compound AI Systems . . . . . | 2        |
| 2.2 手工 Prompting 的问题 . . . . .        | 2        |
| 2.3 本章小结 . . . . .                    | 2        |
| <b>3 DSPy：声明式的 AI 编程</b>              | <b>3</b> |
| 3.1 核心思想 . . . . .                    | 3        |
| 3.2 优化器 (Optimizers) . . . . .        | 3        |
| 3.3 编译 vs. 手写 . . . . .               | 3        |
| 3.4 本章小结 . . . . .                    | 3        |
| <b>4 开放研究方向</b>                       | <b>3</b> |
| <b>5 总结与延伸</b>                        | <b>4</b> |
| 5.1 核心要点 . . . . .                    | 4        |
| 5.2 拓展阅读 . . . . .                    | 4        |

# 1 引言：从单模型到复合 AI 系统

Omar Khattab 是 Databricks 的研究科学家，DSPy 框架的创造者。本讲的核心观点：**单个 LLM 调用不够**——我们需要将多个 LLM 调用、检索、工具组合成**复合 AI 系统** (Compound AI Systems)，并用**编程**而非手工 prompting 来构建它们。

## 从 Demo 到生产

构建一个令人印象深刻的 AI Demo 从未如此容易（如 ChatGPT），但构建一个**可靠**的生产系统仍然很难。核心原因：单次 LLM 调用的能力是有限且不可靠的。

## 1.1 本章小结

复合 AI 系统通过组合多个组件，突破了单次 LLM 调用的能力上限。

# 2 Compound AI Systems 的范式

## 2.1 什么是 Compound AI Systems

### 复合 AI 系统 vs. 单模型

复合 AI 系统由多个相互作用的组件组成：LLM 调用、检索器、代码执行器、验证器等。每个组件有自己的签名（输入/输出类型），通过程序逻辑组合。这类似于传统软件工程中的“函数组合”。

- **RAG** 是最简单的复合系统：检索 + LLM 合成
- **ReAct** 是循环式复合系统：推理 + 工具调用的迭代
- **Multi-hop QA**：多次检索和推理的串联
- 更复杂的系统：多阶段 pipeline，包含验证、反思等步骤

## 2.2 手工 Prompting 的问题

### Prompting 不可持续

手工编写 prompt 看似简单，但在复合系统中会导致：

- **脆弱性**：更换模型或修改 pipeline 需要重写所有 prompt
- **不可组合**：难以将多个 prompt 模块化组合
- **难以优化**：无法系统性地搜索最优 prompt
- **不可移植**：一个模型的好 prompt 对另一个模型可能无效

## 2.3 本章小结

手工 prompting 在复合系统中不可持续，需要更抽象的编程范式。

## 3 DSPy: 声明式的 AI 编程

### 3.1 核心思想

#### DSPy 的核心理念

DSPy 让开发者用 **Python 程序** 定义 AI 系统的结构和逻辑，由框架自动**编译**出最优的 prompt 和 few-shot 示例。开发者只需声明“做什么”（what），不需要手工指定“怎么做”（how，即具体的 prompt 文本）。

关键抽象：

- **Signature**: 定义模块的输入/输出类型（如 “question -> answer”）
- **Module**: 可组合的 AI 组件（如 ChainOfThought、ReAct）
- **Optimizer**: 自动搜索最优 prompt、few-shot 示例和模块配置
- **Metric**: 定义系统质量的评估标准

### 3.2 优化器 (Optimizers)

DSPy 提供多种优化器来自动改进系统：

- **BootstrapFewShot**: 从训练数据中自动挑选最佳 few-shot 示例
- **COPRO**: 自动优化 prompt 中的指令文本
- **MIPRO**: 同时优化指令和示例
- **BootstrapFinetune**: 生成训练数据并 fine-tune 底层模型

### 3.3 编译 vs. 手写

#### 类比传统编程

DSPy 之于 prompt engineering，如同编译器之于汇编语言。开发者用高级抽象编写程序，编译器（optimizer）自动生成底层实现（prompt）。这不仅提高了效率，还使系统具有可移植性——换模型只需重新编译。

### 3.4 本章小结

DSPy 通过声明式编程和自动优化，将复合 AI 系统的构建从手工 prompt engineering 提升为系统化的软件工程。

## 4 开放研究方向

- 更强大的优化器：如何在更大的搜索空间中高效优化？
- 跨模块联合优化：如何同时优化 pipeline 中的多个模块？
- 与 Agent 架构的融合：如何将 DSPy 的优化范式应用于 ReAct 等 Agent 系统？
- 开放生态：鼓励社区贡献新的模块、优化器和评估指标

## 5 总结与延伸

### 5.1 核心要点

1. 复合 AI 系统通过组合多个组件突破单次 LLM 调用的能力限制
2. 手工 prompting 在复合系统中脆弱、不可组合、难以优化
3. DSPy 提供声明式编程范式：定义结构 + 自动编译 prompt
4. Signature、Module、Optimizer、Metric 是四个核心抽象
5. 类比编译器：高级语言（DSPy 程序）→ 底层实现（优化后的 prompt）

### 5.2 拓展阅读

- Khattab et al., “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” ICLR 2024.
- Khattab et al., “Demonstrate-Search-Predict: Composing Retrieval and Language Models for Knowledge-Intensive NLP,” 2023.